



МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РФ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ
«САМАРСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

Институт автоматизации и информационных технологий
Кафедра «Вычислительная техника»

**ГРАФИЧЕСКИЕ
СИСТЕМЫ КОМПЬЮТЕРОВ
НАУЧНО-ИССЛЕДОВАТЕЛЬСКАЯ РАБОТА**
на тему: **Разработка объектно-ориентированного
графического редактора**

Руководитель
доцент кафедры «Вычислительная техника»

Пугачев А. И.

Выполнил обучающийся группы 3-ИАИТ-9

Ершов В.М.

Самара 2021

ОГЛАВЛЕНИЕ

ОГЛАВЛЕНИЕ	2
1. Техническое задание	3
2. Разработка типов данных для представления примитивов и сложных объектов, конструируемых с помощью ТМО.....	4
3. Разработка способов задания параметров примитивов и процедур их визуализации.....	7
4. Программная реализация ТМО.....	19
5. Программная реализация непрерывных геометрических преобразований	23
6. Руководство пользователя.....	27
Библиографический список.....	31

1. Техническое задание

Разработать программу объектно-ориентированного графического редактора, обеспечивающего выполнение следующих основных функций:

- выбор, размещение на экране и визуализация примитивов;
- синтез более сложных фигур с помощью теоретико-множественных операций (ТМО) над примитивами;
- выделение любого объекта, выведенного на экран, и выполнение над ним любой последовательности геометрических преобразований из заданного набора в интерактивном режиме;
- выделение любого объекта на экране и его удаление.

В данной работе были реализованы:

Примитивы:

- Кривая Безье;
- Равнобедренный треугольник
- Флаг

Геометрические преобразования:

- Поворот вокруг центра фигуры на произвольный угол
- Масштабирование по оси X относительно центра фигуры
- Зеркальное отражение относительно прямой общего положения

ТМО:

- Пересечение
- Разность

2. Разработка типов данных для представления примитивов и сложных объектов, конструируемых с помощью ТМО.

Для хранения фигур был создан класс Figure. Переменные класса:

```
public List<PointF> PointList; //список точек фигуры
public Color FigureColor; //цвет фигуры
public bool LineOrCurve = false; //является ли фигура кривой или отрезком
public int TMOfigure = -1; //индекс той фигуры, с которой произошла ТМО
public int TMOtype; //тип ТМО
public bool TMOFirst = false; //была ли выбрана эта фигура первой для ТМО(нужно для ТМО разность)
```

Класс содержит список точек фигуры, цвет фигуры, логическую переменную, определяющую является ли фигура кривой Безье или отрезком прямой, целочисленную переменную, хранящую в себе индекс той фигуры с которой была произведена ТМО у данной фигуры (по умолчанию данное поле равно -1, если оно так и остается равным -1, то с данной фигурой не производилось ТМО), целочисленную переменную определяющую тип производимой ТМО (0 – пересечение, 1 – разность), логическую переменную определяющую была ли эта фигура выбрана первой для ТМО или нет.

Класс имеет два конструктора: один для обычных фигур, второй для отрезков прямой и кривых Безье:

```
public Figure(Color FigureColor)
{
    PointList = new List<PointF>();
    this.FigureColor = FigureColor;
}
public Figure(Color FigureColor, bool LineOrCurve)
{
    PointList = new List<PointF>();
    this.FigureColor = FigureColor;
    this.LineOrCurve = LineOrCurve;
}
```

При создании кривой Безье или отрезка прямой в конструктор вторым параметром передаётся true.

Были написаны методы для работы с объектом класса:

Метод закрашивания фигуры:

```
public void Fill(Graphics g) //метод закрашивания фигуры
{
    int Ymin = (int)PointList[0].Y;
    int Ymax = (int)PointList[0].Y;
    List<int> Xb = new List<int>();
    int k;
    float x;
    for (int i = 0; i < PointList.Count(); i++)
    {
```

```

        if (Ymin > PointList[i].Y)
        {
            Ymin = (int)PointList[i].Y;
        }
        if (Ymax < (int)PointList[i].Y)
        {
            Ymax = (int)PointList[i].Y;
        }
    }
    for (int Y = Ymin; Y <= Ymax; Y++)//для всех строк от Ymin до Ymax
    {
        Xb.Clear();
        for (int i = 0; i < PointList.Count(); i++)//проход по всем точкам
        {
            if (i < PointList.Count() - 1)//если i не последняя точка
            {
                k = i + 1;//k - следующая точка после i
            }
            else k = 0;//иначе k - первая точка
            if (((PointList[i].Y < Y) && (PointList[k].Y >= Y)) || ((PointList[i].Y >=
Y) && (PointList[k].Y < Y)))
            {
                x = (Y - (float)PointList[i].Y) / ((float)PointList[k].Y -
(float)PointList[i].Y) * ((float)PointList[k].X - (float)PointList[i].X) +
(float)PointList[i].X; //нахождение точки пересечения с помощью уравнения
                Xb.Add((int)Math.Round(x));
            }
        }
        Xb.Sort();
        for (int i = 0; i < Xb.Count(); i += 2)
        {
            g.DrawLine(new Pen(FigureColor), Xb[i], Y, Xb[i + 1], Y);
        }
    }
}

```

Данный метод осуществляет построчный проход по Y координатам фигуры (в цикле проверяется каждый Y фигуры от наименьшего к наибольшему) и вычисление границ фигуры по оси X на данном Y, после вычисления границ по оси X происходит закрашивание сегмента с помощью метода DrawLine класса Graphics (в DrawLine передается объект Pen, которым будет происходить закрашивание, координата точки начала закрашиваемой прямой [левый X фигуры; текущий Y] и координата точки конца закрашиваемой прямой [правый X фигуры; текущий Y]).

Метод, который определяет принадлежность точки к фигуре (на его основе был сделан метод выделения фигуры):

```

public bool ThisFigure(int mouseX, int mouseY)//метод, определяющий принадлежит ли
заданная точка фигуре
{
    int n = PointList.Count() - 1, k;
    PointF Pi, Pk;
    double x;
    List<int> Xb = new List<int>(); // буфер сегментов
    bool check = false;

```

```

Xb.Clear();
for (int i = 0; i <= n; i++)
{
    if (i < n) k = i + 1; else k = 0;
    Pi = PointList[i];
    Pk = PointList[k];
    if ((Pi.Y < mouseY) & (Pk.Y >= mouseY) | (Pi.Y >= mouseY) & (Pk.Y < mouseY))
    {
        x = (mouseY - Pi.Y) * (Pk.X - Pi.X) / (Pk.Y - Pi.Y) + Pi.X;
        Xb.Add((int)Math.Round(x));
    }
}
if (Xb.Count() > 0)
{
    Xb.Sort(); // сортировка по возрастанию
    for (int i = 0; i < Xb.Count; i += 2)
        if (mouseX >= Xb[i] & mouseX <= Xb[i + 1]) { check = true; break; }
}
return check;
}

```

Данный метод был сделан на основе алгоритма закрашивания фигуры, только вместо закрашивания осуществляется проверка: принадлежит ли заданная точка вычисленной границе (сначала проверяется входит ли точка в границы по Y фигуры, затем входит ли по X), если принадлежит возвращается true, иначе цикл повторяется вновь, если программа определит, что точка не принадлежит ни одной из границ фигуры, она вернет false.

Следующий метод ищет центральную точку фигуры (на него ссылаются методы для геометрических операций, выполняемых относительно центра примитива):

```

public Point FindCenter()//метод поиск центральной точки
{
    Point C = new Point();
    int xMax = 0, xMin = (int)Math.Round(PointList[0].X), yMax = 0, yMin =
(int)Math.Round(PointList[0].Y);
    for (int i = 0; i < PointList.Count; i++)//находим максимальные и минимальные
вершины
    {
        if (PointList[i].X > xMax) xMax = (int)Math.Round(PointList[i].X);
        else if (PointList[i].X < xMin) xMin = (int)Math.Round(PointList[i].X);
        if (PointList[i].Y > yMax) yMax = (int)Math.Round(PointList[i].Y);
        else if (PointList[i].Y < yMin) yMin = (int)Math.Round(PointList[i].Y);
    }
    C.X = xMin + ((xMax - xMin) / 2);//находим центральную точку
    C.Y = yMin + ((yMax - yMin) / 2);
    return C;
}

```

Данный метод осуществляет поиск максимальных X и Y фигуры и поиск минимальных X и Y фигуры, затем используя эти координаты в формуле, происходит вычисление координат точки центра фигуры.

3. Разработка способов задания параметров примитивов и процедур их визуализации.

Переменные класса Form1:

```
Graphics g;
Bitmap myBitmap;
List<PointF> VertexList = new List<PointF>(); //список точек одной фигуры
List<Figure> ListOfFigures = new List<Figure>(); //список фигур
int PointCounter, FigureCounter;
Point pictureBox1MousePosition = new Point();
Pen DrawPen = new Pen(Color.Black);
int PrimitiveType, GeometricOperationType, TMOtype;
int mode = 0; //1-рисование, 2-геометрические операции, 3-ТМО
bool rightclick = false; //нужно для отрисовки кривой Безье
bool checkPrimitive = false; //показывает, выделен ли примитив
int Current; //индекс выделенного примитива
Point Center; //координаты центра выделенного примитива
Pen Xpen = new Pen(Color.Red);
bool Figure1captured = false; //переменная для реализации ТМО
int Figure1index, Figure2index; //индекс первой и второй фигур, участвующих в ТМО
int MirrorPointCount = 0;
Point MirrorPoint1, MirrorPoint2 = new Point(); //координаты двух точек прямой общего
положения
```

Загрузчик формы:

```
public Form1()
{
    InitializeComponent();
    myBitmap = new Bitmap(pictureBox1.Width, pictureBox1.Height);
    g = Graphics.FromImage(myBitmap);
}
```

Пользователь может взаимодействовать с тремя комбо-боксами: Примитивы, Преобразования и ТМО. При этом, комбо-бкс с геометрическими преобразованиями становится доступен когда введена хотя бы одна фигура, а комбо-бкс с ТМО, если введено как минимум две.

Реализация выбора примитива:

```
private void comboBox1_SelectedIndexChanged(object sender, EventArgs e) //обработчик
изменения "Примитивы"
{
    comboBox2.Text = "Преобразования";
    comboBox3.Text = "ТМО";
    PrimitiveType = comboBox1.SelectedIndex;
    mode = 1;
    Figure1captured = false; //отмена выделения фигуры для ТМО
    MirrorPointCount = 0; //отмена операции отражения
    VertexList.Clear();
    PointCounter = 0;
    g.Clear(pictureBox1.BackColor);
    DrawAllFigures(ListOfFigures);
    pictureBox1.Image = myBitmap;
}
```

Главные функции данного комбо-бокса – это включение режима рисование (mode = 1) и запоминание индекса выбранного примитива (0 – кривая Безье, 1 – равнобедренный треугольник, 2 – флаг, 3 – отрезок прямой).

Реализация выбора преобразования:

```
private void comboBox2_SelectedIndexChanged(object sender, EventArgs e)//обработчик
изменения "Преобразования"
{
    if (FigureCounter == 0)
    {
        comboBox2.Text = "Преобразования";
        MessageBox.Show($"Сначала нарисуйте хотябы одну фигуру");
        mode = 0;
    }
    else
    {
        GeometricOperationType = comboBox2.SelectedIndex;
        mode = 2;
    }
    comboBox1.Text = "Примитивы";
    comboBox3.Text = "ТМО";
    Figure1captured = false;
    MirrorPointCount = 0;
    VertexList.Clear();
    PointCounter = 0;
    g.Clear(pictureBox1.BackColor);
    DrawAllFigures(ListOfFigures);
    pictureBox1.Image = myBitmap;
}
```

Главные функции данного комбо-бокса – это включение режима геометрических операций (mode = 2) и запоминание индекса выбранной операции (0 – перемещение, 1 – поворот, 2 – масштабирование, 3 – отражение).

Реализация выбора ТМО:

```
private void comboBox3_SelectedIndexChanged(object sender, EventArgs e)//обработчик
изменения "ТМО"
{
    if (FigureCounter < 2)
    {
        comboBox3.Text = "ТМО";
        MessageBox.Show($"Сначала нарисуйте хотябы две фигуры");
        mode = 0;
    }
    else
    {
        TMOtype = comboBox3.SelectedIndex;
        mode = 3;
    }
    comboBox1.Text = "Примитивы";
    comboBox2.Text = "Преобразования";
    Figure1captured = false;
    MirrorPointCount = 0;
    VertexList.Clear();
    PointCounter = 0;
    g.Clear(pictureBox1.BackColor);
    DrawAllFigures(ListOfFigures);
    pictureBox1.Image = myBitmap;
}
```



```
}
```

Главные функции данного комбо-бокса – это включение режима ТМО (mode = 2) и запоминание индекса выбранной ТМО (0 – пересечение, 1 – разность).

Для выбора цвета вызывается диалоговое окно (нажатием кнопки «Цвет»):

```
private void ColorChoice(object sender, EventArgs e)//обработчик кнопки "Цвет"
{
    colorDialog1.ShowDialog();
    DrawPen.Color = colorDialog1.Color;
}
```

Задание точек примитивов происходит с помощью нажатия кнопок мыши, геометрические преобразования происходят при передвижении мыши и при нажатии кнопок мыши, ТМО происходят при нажатии кнопок мыши. Рассмотрим обработчики данных событий:

Обработчик нажатия кнопок мыши:

```
private void pictureBox1_MouseDown(object sender, MouseEventArgs e)//обработчик
нажатия кнопок мыши в области pictureBox1
{
    pictureBox1MousePosition = e.Location;
    if (checkBox1.Checked)//если включен режим удаления
    {
        if (e.Button == MouseButton.Right)//фигуры удалятся при нажатии ПКМ по ним
        {
            int n = SelectPrimitive(pictureBox1MousePosition);//индекс фигуры
            if (checkPrimitive)//если фигура выбрана
            {
                if (ListOfFigures[n].TMOfigure != -1)//если фигура участвует в ТМО
                {
                    int f = ListOfFigures[n].TMOfigure;
                    if (n > ListOfFigures[n].TMOfigure)//если индекс выбранной фигуры
большее индекса связанной с ней фигурой
                    {
                        ListOfFigures.RemoveAt(ListOfFigures[n].TMOfigure);//удаление
связанной фигуры

                        ListOfFigures.RemoveAt(n - 1);//удаление выбранной фигуры
                        for (int i = 0; i < ListOfFigures.Count; i++)//изменение поля
ТМОfigure у всех других фигур, участвующих в ТМО и изменивших свой индекс в результате
удаления
                        {
                            if (ListOfFigures[i].TMOfigure != -1)
                            {
                                if (ListOfFigures[i].TMOfigure > f &
ListOfFigures[i].TMOfigure < n) ListOfFigures[i].TMOfigure -= 1;
                                else if (ListOfFigures[i].TMOfigure > f &
ListOfFigures[i].TMOfigure > n) ListOfFigures[i].TMOfigure -= 2;
                            }
                        }
                    }
                }
                else//если индекс выбранной фигуры меньше индекса связанной с ней
фигурой
                {
                    ListOfFigures.RemoveAt(ListOfFigures[n].TMOfigure);
                    ListOfFigures.RemoveAt(n);
                }
            }
        }
    }
}
```

```

        for (int i = 0; i < ListOfFigures.Count; i++)
        {
            if (ListOfFigures[i].TMOfigure != -1)
            {
                if (ListOfFigures[i].TMOfigure > n &
                ListOfFigures[i].TMOfigure < f) ListOfFigures[i].TMOfigure -= 1;
                else if (ListOfFigures[i].TMOfigure > n &
                ListOfFigures[i].TMOfigure > f) ListOfFigures[i].TMOfigure -= 2;
            }
        }
        FigureCounter -= 2;
    }
    else//если удаление фигуры, не участвующей в ТМО
    {
        ListOfFigures.RemoveAt(n);
        for (int i = 0; i < ListOfFigures.Count; i++)//изменение поля
        TMOfigure у всех других фигур, участвующих в ТМО и изменивших свой индекс в результате
        удаления
        {
            if (ListOfFigures[i].TMOfigure != -1)
            {
                if (ListOfFigures[i].TMOfigure > n)
                ListOfFigures[i].TMOfigure -= 1;
            }
        }
        FigureCounter--;
    }
    g.Clear(pictureBox1.BackColor);
    DrawAllFigures(ListOfFigures);
}
}
else
{
    if (mode == 1)//если включен режим рисования
    {
        if (e.Button == MouseButtons.Right)
            rightclick = true;
        else
            g.DrawEllipse(DrawPen, e.X - 2, e.Y - 2, 5, 5);
            DrawPrimitive();
    }
    else if (mode == 2)// если включен режим преобразования
    {
        if (GeometricOperationType == 3)//если выбрано отражение
        {
            if (e.Button == MouseButtons.Right)
            {
                switch (MirrorPointCount)
                {
                    case 0:
                    {
                        MirrorPoint1 = new Point(e.X, e.Y);//первая точка
                        прямой общего положения
                        g.DrawEllipse(DrawPen, e.X - 2, e.Y - 2, 5, 5);
                        MirrorPointCount++;
                        break;
                    }
                    case 1:
                    {
                        MirrorPoint2 = new Point(e.X, e.Y);//вторая точка
                        общего положения
                        if (ListOfFigures[Current].TMOfigure != -1)//если
                        фигура участвует в ТМО

```

```

ListOfFigures[ListOfFigures[Current].TMOfigure].Mirror(MirrorPoint1,
MirrorPoint2);//отражается связанная с ней фигура
ListOfFigures[Current].Mirror(MirrorPoint1,
MirrorPoint2);//отражается сама фигура
g.Clear(pictureBox1.BackColor);
g.DrawEllipse(DrawPen, MirrorPoint1.X, MirrorPoint1.Y,
5, 5);//зарисовывается прямая общего положения
g.DrawEllipse(DrawPen, MirrorPoint2.X, MirrorPoint2.Y,
5, 5);

g.DrawLine(DrawPen, MirrorPoint1, MirrorPoint2);
DrawAllFigures(ListOfFigures);
MirrorPointCount = 0;
break;
    }
    }
}
if (e.Button == MouseButtons.Left)// находим индекс и центр фигуры,
выбранной пользователем(выбор осуществляется нажатием лкм в зоне фигуры)
{
    int n = SelectPrimitive(pictureBox1.MousePosition);
    if (checkPrimitive)
    {
        Current = n;//запоминается индекс выбранной фигуры
        Center = ListOfFigures[Current].FindCenter();//находится центр
        MirrorPointCount = 0;//отмена операции отражения
        g.Clear(pictureBox1.BackColor);
        DrawAllFigures(ListOfFigures);
        DrawX(pictureBox1.MousePosition);//зарисовка перекрестия
    }
}
else if (mode == 3)// если включен режим ТМО
{
    if (e.Button == MouseButtons.Left)
    {
        if (!Figure1captured)//если первая фигура не выбрана
        {
            Figure1index = SelectPrimitive(pictureBox1.MousePosition);
            if (checkPrimitive)
            {
                if (ListOfFigures[Figure1index].TMOfigure != -1)//если
                пользователь пытается выбрать результат ТМО в качестве операнда ТМО
                {
                    MessageBox.Show($"С результатом ТМО нельзя проводить
ТМО");
                }
                else if (ListOfFigures[Figure1index].LineOrCurve)//если
                пользователь пытается выбрать кривую или отрезок в качестве операнда ТМО
                {
                    MessageBox.Show($"С данным примитивом нельзя проводить
ТМО");
                }
                else
                {
                    Figure1captured = true;
                    DrawX(pictureBox1.MousePosition);//зарисовка перекрестия
                }
            }
        }
        else//если первая фигура выбрана
        {
            Figure2index = SelectPrimitive(pictureBox1.MousePosition);

```

```

        if (checkPrimitive)
        {
            if (ListOfFigures[Figure2index].TMOfigure != -1)
            {
                MessageBox.Show($"С результатом ТМО нельзя проводить
TMO");
            }
            else if (ListOfFigures[Figure2index].LineOrCurve)
            {
                MessageBox.Show($"С данным примитивом нельзя проводить
TMO");
            }
            else if (Figure1index == Figure2index)//если пользователь
пытается выбрать в качестве второй фигуры первую фигуру
            {
                MessageBox.Show($"Выберите два разных примитива");
            }
            else
            {
                ListOfFigures[Figure1index].FigureColor =
DrawPen.Color;//задание цвета результату ТМО
                ListOfFigures[Figure1index].TMOfigure = Figure2index;//у
первой фигуры запоминается индекс второй
                ListOfFigures[Figure1index].TMOtype =
TMOtype;//запоминание типа ТМО
                ListOfFigures[Figure1index].TMOFirst = true;//обозначение
того, какая фигура была выбрана первой
                ListOfFigures[Figure2index].FigureColor =
DrawPen.Color;//задание цвета результату ТМО
                ListOfFigures[Figure2index].TMOfigure = Figure1index;//у
второй фигуры запоминается индекс первой
                Figure1captured = false;
                g.Clear(pictureBox1.BackColor);
                DrawAllFigures(ListOfFigures);
            }
        }
    }
}
}
}
pictureBox1.Image = myBitmap;
}
}

```

С включенным режимом удаления при нажатии правой кнопки мыши (ПКМ) в области любой фигуры, произойдет ее удаление. При условии, что выбран один из примитивов, нажатия левой кнопки мыши (и правой в случае выбора кривой Безье) влекут за собой вызов метода DrawPrimitive, благодаря которому происходит создание примитивов (данный метод будет рассмотрен ниже). При условии, что выбрано одно из геометрических преобразований, нажатие ЛКМ по примитиву выделяет его (все геометрические преобразования происходят только с выделенным примитивом и по умолчанию выделен первый нарисованный примитив), если выбрано геометрическое преобразование «Отражение», то первое нажатие ПКМ задает первую точку отрезка прямой общего положения, второе нажатие ПКМ задает вторую точку отрезка прямой

общего положения и вызывает отражение выделенной фигуры относительно заданной прямой. При условии, что выбрана одна из ТМО, нажатие ЛКМ по фигуре выделит ее, когда будет выполнено выделение двух фигур, над ними произойдет выбранная ТМО.

Обработчик движения мыши:

```
private void pictureBox1_MouseMove(object sender, MouseEventArgs e)//обработчик
движения мыши в области pictureBox1
{
    if (mode == 2)// если включен режим преобразования
    {
        switch (GeometricOperationType)
        {
            case 0://Перемещение
            {
                if (e.Button == MouseButtons.Left)
                {
                    if (ListOfFigures[Current].TMOfigure != -1)//если выбранная
фигура участвует в ТМО
                        ListOfFigures[ListOfFigures[Current].TMOfigure].Move(e.X -
pictureBox1MousePosition.X, e.Y - pictureBox1MousePosition.Y);//двигается связанная с ней
фигура
                        ListOfFigures[Current].Move(e.X - pictureBox1MousePosition.X,
e.Y - pictureBox1MousePosition.Y);//двигается выбранная фигура
                        g.Clear(pictureBox1.BackColor);
                        DrawAllFigures(ListOfFigures);
                        pictureBox1.Image = myBitmap;
                        pictureBox1MousePosition = e.Location;
                }
                break;
            }
            case 1://Поворот
            {
                if (e.Button == MouseButtons.Left)
                {
                    if (ListOfFigures[Current].TMOfigure != -1)//если выбранная
фигура участвует в ТМО
                        ListOfFigures[ListOfFigures[Current].TMOfigure].Turn(pictureBox1MousePosition, new Point(e.X,
e.Y), Center);//поворачивается связанная с ней фигура
                        ListOfFigures[Current].Turn(pictureBox1MousePosition, new
Point(e.X, e.Y), Center);//поворачивается выбранная фигура
                        g.Clear(pictureBox1.BackColor);
                        DrawAllFigures(ListOfFigures);
                        pictureBox1.Image = myBitmap;
                        pictureBox1MousePosition = e.Location;
                }
                break;
            }
            case 2://Масштабирование
            {
                if (e.Button == MouseButtons.Left)
                {
                    if (ListOfFigures[Current].TMOfigure != -1)//если выбранная
фигура участвует в ТМО
                        ListOfFigures[ListOfFigures[Current].TMOfigure].Scale(pictureBox1MousePosition, new Point(e.X,
e.Y), Center);//масштабируется связанная с ней фигура
                        ListOfFigures[Current].Scale(pictureBox1MousePosition, new
Point(e.X, e.Y), Center);//масштабируется выбранная фигура
```

```

        g.Clear(pictureBox1.BackColor);
        DrawAllFigures(ListOfFigures);
        pictureBox1.Image = myBitmap;
        pictureBox1.MousePosition = e.Location;
    }
    break;
}
}
}
}
}

```

Геометрические преобразования перемещение, поворот и масштабирование реализованы следующим образом: пользователь выделяет фигуру нажатием ЛКМ, затем, не отпуская ЛКМ, передвигает мышь. В случае перемещения фигура будет перемещаться в след за курсором, в случае поворота фигура будет поворачиваться вокруг своего центра на тот угол, на который поворачивается курсор, в случае масштабирования фигура будет изменять свою ширину с изменением расстояния между курсором и центром фигуры (чем больше расстояние, тем шире фигура и наоборот, чем меньше расстояние, тем фигура уже).

Метод DrawPrimitive:

```

public void DrawPrimitive()//функция создания примитивов
{
    switch (PrimitiveType)
    {
        case 0://Кривая Безье
        {
            if (!rightclick)//пока не нажата ПКМ
            {
                VertexList.Add(pictureBox1.MousePosition);//запоминаются точки
            }
            else
            {
                VertexList.Add(pictureBox1.MousePosition);//запоминается точка ПКМ
                ListOfFigures.Add(new Figure(DrawPen.Color, true));//в список
                фигур добавляется новая фигура
                ListOfFigures[FigureCounter].PointList.AddRange(VertexList);//у
                только что добавленной фигуры заполняется список точек
                FigureCounter++;//счетчик фигур
                rightclick = false;
                g.Clear(pictureBox1.BackColor);
                DrawAllFigures(ListOfFigures);
                VertexList.Clear();
            }
            break;
        }
        case 1://Равнобедренный треугольник
        {
            switch (PointCounter)
            {
                case 0:
                {
                    VertexList.Add(pictureBox1.MousePosition);//добавляется
                    первая точка
                    PointCounter++;
                    break;
                }
            }
        }
    }
}

```

```

        case 1:
        {
            VertexList.Add(pictureBox1MousePosition); //добавляется
вторая точка
            int dx = (int)(VertexList[1].X - VertexList[0].X); //по
имеющимся координатам первых двух точек
            VertexList.Add(new PointF(VertexList[1].X + dx,
VertexList[0].Y)); //создается третья точка
            VertexList.Add(VertexList[0]); //в конец списка фигур
добавляется первая точка(для реализации ТМО)
            ListOfFigures.Add(new Figure(DrawPen.Color));

ListOfFigures[FigureCounter].PointList.AddRange(VertexList);
            FigureCounter++;
            g.Clear(pictureBox1.BackColor);
            DrawAllFigures(ListOfFigures);
            VertexList.Clear();
            PointCounter = 0;
            break;
        }
    }
    break;
}
case 2: //Флаг
{
    switch (PointCounter)
    {
        case 0:
        {
            VertexList.Add(pictureBox1MousePosition); //добавление
первой точки
            PointCounter++;
            break;
        }
        case 1:
        {
            int dx = (int)(pictureBox1MousePosition.X -
((pictureBox1MousePosition.X - VertexList[0].X) / 3)); //координаты для выреза на флажке
            int dy = (int)(VertexList[0].Y +
((pictureBox1MousePosition.Y - VertexList[0].Y) / 2));
            VertexList.Add(new PointF(VertexList[0].X,
pictureBox1MousePosition.Y)); //по имеющимся координатам первых двух точек
            VertexList.Add(pictureBox1MousePosition); //создаются
остальные точки
            VertexList.Add(new Point(dx, dy));
            VertexList.Add(new PointF(pictureBox1MousePosition.X,
VertexList[0].Y));
            VertexList.Add(VertexList[0]);
            ListOfFigures.Add(new Figure(DrawPen.Color));

ListOfFigures[FigureCounter].PointList.AddRange(VertexList);
            FigureCounter++;
            g.Clear(pictureBox1.BackColor);
            DrawAllFigures(ListOfFigures);
            VertexList.Clear();
            PointCounter = 0;
            break;
        }
    }
    break;
}
case 3: //Отрезок прямой
{
    switch (PointCounter)
    {

```

```

        case 0:
        {
            VertexList.Add(pictureBox1.MousePosition);
            PointCounter++;
            break;
        }
        case 1:
        {
            VertexList.Add(pictureBox1.MousePosition);
            ListOfFigures.Add(new Figure(DrawPen.Color, true));

            ListOfFigures[FigureCounter].PointList.AddRange(VertexList);
            FigureCounter++;
            g.Clear(pictureBox1.BackColor);
            DrawAllFigures(ListOfFigures);
            VertexList.Clear();
            PointCounter = 0;
            break;
        }
    }
    break;
}
}
}

```

Работа данного метода построена на операторе switch-case. Для кривой Безье он записывает точки (они задаются нажатием ЛКМ), после нажатия ПКМ задается последняя точка и в функции DrawAllFigures вызывается отдельный метод инициализации кривой. Инициализацию треугольника, флага и отрезка прямой он содержит непосредственно в себе, ввиду того, что для этого требуются гораздо более простые алгоритмы. Для равнобедренного треугольника метод записывает две точки, которые становятся одной из равнобедренных сторон. После этого, используя координаты заданных двух точек задается третья, таким образом, что по ним создается равнобедренный треугольник. Достигается это простейшими геометрическими правилами. Все точки треугольника заносятся в список точек фигуры. Флаг рисуется похожим образом – вводятся две точки, только у флага они являются вершинами диагонали фигуры, в остальном алгоритмы очень похожи. Отрезок прямой рисуется по двум точкам.

Метод DrawAllFigures:

```

private void DrawAllFigures(List<Figure> Figures)//метод зарисовки всех фигур
{
    for (int i = 0; i < Figures.Count(); i++)//проход по списку фигур
    {
        if (ListOfFigures[i].TMOfigure != -1)//если фигура участвует в ТМО
        {
            if (Figures[i].TMOFirst)//если фигура первая
                TMO(Figures[i].TMOtype, Figures[i],
Figures[Figures[i].TMOfigure]); //происходит ТМО
            else DrawFigure(Figures[i]); //зарисовка фигуры
        }
    }
}

```



```
    }
}
```

Данный метод проходит по списку фигур и если текущая фигура участвует в ТМО и является первой выбранной для ТМО, то происходит зарисовка полученной в результате ТМО области, иначе если фигура не участвует в ТМО, то вызывается метод DrawFigure.

Метод DrawFigure:

```
private void DrawFigure(Figure Figure)//метод зарисовки фигуры
{
    if (Figure.LineOrCurve)//если фигура это отрезок прямой или кривая
    {
        if (Figure.PointList.Count() == 2)//Отрезок прямой
        {
            g.DrawLine(new Pen(Figure.FigureColor), Figure.PointList[0],
Figure.PointList[1]);
        }
        else//Кривая Безье
        {
            DrawBezier(new Pen(Figure.FigureColor), Figure.PointList,
Figure.PointList.Count() - 1);
        }
    }
    else//Треугольник или флаг
    {
        Figure.Fill(g);
    }
}
```

Данный метод зарисовывает переданную ему в качестве параметра фигуру, при чем, если поле LineOrCurve этой фигуры равно true, то по количеству точек определяется отрезок прямой это или кривая Безье, если это отрезок, то он зарисовывается методом DrawLine класса Graphics, если это кривая Безье то она зарисовывается методом DrawBezier из лабораторной работы №1. Если поле LineOrCurve переданной фигуры равно false, то вызывается метод класса Figure под названием Fill, описанный выше.

Метод выделения фигуры:

```
public int SelectPrimitive(Point pictureBox1MousePos)//выбор фигуры (возвращает ее
индекс в списке фигур и задает checkPrimitive == true)
{
    int n = ListOfFigures.Count + 1;
    for (int i = ListOfFigures.Count - 1; i >= 0; i--)//проход по списку фигур,
начиная с конца
    {
        if (ListOfFigures[i].ThisFigure(pictureBox1MousePos.X,
pictureBox1MousePos.Y))//если заданная точка принадлежит фигуре
        {
            checkPrimitive = true;
            n = i;//запоминается индекс этой фигуры
            break;
        }
    }
}
```

```

    }
    if (n == ListOfFigures.Count + 1)//если заданная точка не принадлежит ни одной
фигуре
    {
        checkPrimitive = false;
        MessageBox.Show($"Укажите точку, лежащую на одном из примитивов");
        return n;
    }
    else return n;
}

```

Данный метод проверяет наличие выбранной точки в каждом объекте, начиная с последнего, пока не найдёт первое совпадение, после чего возвращает индекс выбранной фигуры, с которым в дальнейшем выполняются операции, или в случае не нахождения совпадения выводится сообщение.

4. Программная реализация ТМО

Реализация ТМО содержится в Form1:

```
private void ТМО(int type, Figure Figure1, Figure Figure2)
{
    List<int> Xal = new List<int>(), Xar = new List<int>();//список левых и список
    List<int> Xbl = new List<int>(), Xbr = new List<int>();//список левых и список
    List<int> Yal = new List<int>(), Yar = new List<int>();//список левых и список
    List<int> Ybl = new List<int>(), Ybr = new List<int>();//список левых и список
    int YminA = (int)Figure1.PointList[0].Y;//нижняя граница фигуры A
    int YmaxA = (int)Figure1.PointList[0].Y;//верхняя граница фигуры A
    int YminB = (int)Figure2.PointList[0].Y;//нижняя граница фигуры B
    int YmaxB = (int)Figure2.PointList[0].Y;//верхняя граница фигуры B
    int ko;
    float xF;//вычисленный икс
    int X;//округленный вычисленный икс
    int AindexMin = 1;//индекс точки, являющейся нижней границей фигуры A
    bool aCW;//обход фигуры A
    int BindexMin = 1;//индекс точки, являющейся нижней границей фигуры B
    bool bCW;//обход фигуры B

    for (int i = 1; i < Figure1.PointList.Count()-1; i++)
    {
        if (YminA > Figure1.PointList[i].Y)
        {
            YminA = (int)Figure1.PointList[i].Y;
            AindexMin = i;
        }
        if (YmaxA < Figure1.PointList[i].Y)
        {
            YmaxA = (int)Figure1.PointList[i].Y;
        }
    }
    for (int i = 1; i < Figure2.PointList.Count()-1; i++)
    {
        if (YminB > Figure2.PointList[i].Y)
        {
            YminB = (int)Figure2.PointList[i].Y;
            BindexMin = i;
        }
        if (YmaxB < Figure2.PointList[i].Y)
        {
            YmaxB = (int)Figure2.PointList[i].Y;
        }
    }
    //вычисление обхода фигуры A
    if ((Figure1.PointList[AindexMin - 1].X * (Figure1.PointList[AindexMin].Y -
    Figure1.PointList[AindexMin + 1].Y) + Figure1.PointList[AindexMin].X *
    (Figure1.PointList[AindexMin + 1].Y - Figure1.PointList[AindexMin - 1].Y) +
    Figure1.PointList[AindexMin + 1].X * (Figure1.PointList[AindexMin - 1].Y -
    Figure1.PointList[AindexMin].Y)) > 0)//формула площади треугольника
        aCW = true;//если площадь треугольника > 0 обход фигуры по часовой стрелке
    else aCW = false;//иначе против часовой

    //вычисление обхода фигуры B
    if ((Figure2.PointList[BindexMin - 1].X * (Figure2.PointList[BindexMin].Y -
    Figure2.PointList[BindexMin + 1].Y) + Figure2.PointList[BindexMin].X *
    (Figure2.PointList[BindexMin + 1].Y - Figure2.PointList[BindexMin - 1].Y) +
    Figure2.PointList[BindexMin + 1].X * (Figure2.PointList[BindexMin - 1].Y -
    Figure2.PointList[BindexMin].Y)) > 0)
        bCW = true;
    else bCW = false;
```

```

        for (int Y = Math.Min(YminA, YminB); Y <= Math.Max(YmaxA, YmaxB); Y++)//для всех
        строк от Ymin до Ymax
        {
            Xal.Clear();
            Xar.Clear();
            for (int i = 0; i < Figure1.PointList.Count(); i++)//проход по всем точкам
            фигуры A
            {
                if (i < Figure1.PointList.Count() - 1)//если i не последняя точка
                {
                    ko = i + 1;//ко - следующая точка после i
                }
                else ko = 0;//иначе ко - первая точка
                if (((Figure1.PointList[i].Y < Y) && (Figure1.PointList[ko].Y >= Y)) ||
                ((Figure1.PointList[i].Y >= Y) && (Figure1.PointList[ko].Y < Y)))
                {
                    xF = ((float)Y - (float)Figure1.PointList[i].Y) /
                ((float)Figure1.PointList[ko].Y - (float)Figure1.PointList[i].Y) *
                ((float)Figure1.PointList[ko].X - (float)Figure1.PointList[i].X) +
                (float)Figure1.PointList[i].X; //нахождение икса точки пересечения строки Y со стороной фигуры
                А с помощью уравнения
                    X = (int)Math.Round(xF);
                    if (aCW)//если по часовой
                    {
                        if (Figure1.PointList[ko].Y - Figure1.PointList[i].Y > 0) Xar.Add(X);
                        //заполнение списка "правых" иксов фигуры А
                    }
                    else Xal.Add(X); //заполнение списка "левых" иксов фигуры А
                    }
                    else if (!aCW)
                    {
                        if (Figure1.PointList[ko].Y - Figure1.PointList[i].Y < 0)
                        Xar.Add(X);
                        else Xal.Add(X);
                    }
                }
            }
            Xal.Sort();//сортировка по возрастанию
            Xar.Sort();

            //все тоже самое для фигуры В
            Xbl.Clear();
            Xbr.Clear();
            for (int i = 0; i < Figure2.PointList.Count(); i++)
            {
                if (i < Figure2.PointList.Count() - 1)
                {
                    ko = i + 1;
                }
                else ko = 0;
                if (((Figure2.PointList[i].Y < Y) && (Figure2.PointList[ko].Y >= Y)) ||
                ((Figure2.PointList[i].Y >= Y) && (Figure2.PointList[ko].Y < Y)))
                {
                    xF = ((float)Y - (float)Figure2.PointList[i].Y) /
                ((float)Figure2.PointList[ko].Y - (float)Figure2.PointList[i].Y) *
                ((float)Figure2.PointList[ko].X - (float)Figure2.PointList[i].X) +
                (float)Figure2.PointList[i].X;
                    X = (int)Math.Round(xF);
                    if (bCW)
                    {
                        if (Figure2.PointList[ko].Y - Figure2.PointList[i].Y > 0) Xbr.Add(X);
                        else Xbl.Add(X);
                    }
                    else if (!bCW)
                    {

```

```

        if (Figure2.PointList[ko].Y - Figure2.PointList[i].Y < 0)
            Xbr.Add(X);
        else Xbl.Add(X);
    }
}
Xbl.Sort();
Xbr.Sort();

List<int> Mx = new List<int>(); //список для записи координаты x границы
сегмента
int[] MdQ = new int[Xal.Count() + Xar.Count() + Xbl.Count() +
Xbr.Count()]; //массив для записи соответствующего приращения пороговой функции с учетом веса
операнда
int[] SetQ = new int[2]; //множество значений суммы Q пороговых функций
операндов, соответствующее заданной ТМ0;
int nM, n;
List<int> Xr1 = new List<int>(), Xr2 = new List<int>(); //Результатом работы
алгоритма будут массивы Xr1 и Xr2 левых и правых границ сегментов сечения результирующей
области строкой Y.
if (type == 0) { SetQ[0] = 3; SetQ[1] = 3; } //пересечение
else if (type == 1) { SetQ[0] = 2; SetQ[1] = 2; } //разность
n = Xal.Count();
for (int i = 1; i <= n; i++)
{
    Mx.Add(Xal[i - 1]);
    MdQ[i - 1] = 2;
}
nM = n;
n = Xar.Count();
for (int i = 1; i <= n; i++)
{
    Mx.Add(Xar[i - 1]);
    MdQ[nM + i - 1] = -2;
}
nM += n;
n = Xbl.Count();
for (int i = 1; i <= n; i++)
{
    Mx.Add(Xbl[i - 1]);
    MdQ[nM + i - 1] = 1;
}
nM += n;
n = Xbr.Count();
for (int i = 1; i <= n; i++)
{
    Mx.Add(Xbr[i - 1]);
    MdQ[nM + i - 1] = -1;
}
nM += n; // общее число элементов в массиве Mx

for (int i = 0; i < nM; i++) //сортировка списка Mx по возрастанию и массива
MdQ относительно массива Mx
{
    for (int j = 0; j < nM - 1 - i; j++)
    {
        if (Mx[j] > Mx[j + 1])
        {
            int buff1 = Mx[j];
            int buff2 = MdQ[j];

            Mx[j] = Mx[j + 1];
            MdQ[j] = MdQ[j + 1];

            Mx[j + 1] = buff1;

```

```

        MdQ[j + 1] = buff2;
    }
}

int Q = 0;
for (int i = 0; i < nM; i++)//проход по всему списку Mx
{
    int x = Mx[i];
    int Qnew = Q + MdQ[i];
    if (!(Q >= SetQ[0] && Q <= SetQ[1]) && (Qnew >= SetQ[0] && Qnew <=
SetQ[1]))
    {
        Xr1.Add(x);
    }
    if ((Q >= SetQ[0] && Q <= SetQ[1]) && !(Qnew >= SetQ[0] && Qnew <=
SetQ[1]))
    {
        Xrr.Add(x);
    }
    Q = Qnew;
}

for (int i = 0; i < Xr1.Count(); i++)//закраска результирующей области
{
    g.DrawLine(new Pen(Figure1.FigureColor), Xr1[i], Y, Xrr[i], Y);
}
}

```

В качестве параметров данному методу передается тип ТМО (0 = пересечение, 1 = разность), и две фигуры с которыми и происходит операция. Сначала определяются границы по Y, в которых будут происходить вычисления (от наименьшего игрека среди двух фигур, до наибольшего игрека среди двух фигур), затем, на каждом из игреков в вычисленном промежутке ищутся границы по X для двух фигур, после чего они записываются в один список и упорядочиваются по возрастанию, затем, на основании выбранной ТМО определяются границы по X результирующей области на текущем Y и происходит их закрашивание.

5. Программная реализация непрерывных геометрических преобразований

Методы для выполнения геометрических операций были написаны в классе Figure. Когда пользователь совершает такую операцию, метод вызывается из объекта, с которым проводится операция.

Функция, используемая для матричной реализации геометрических преобразований:

```
private void SameAction(double[,] matrix1, double[,] matrix2)//данные действия выполнялись в каждом геометрическом преобразовании, поэтому были вынесены в отдельную функцию
{
    int n = PointList.Count();
    PointF fP = new Point();
    double[,] result;
    for (int i = 0; i < n; i++)
    {
        matrix1[0, 0] = PointList[i].X; matrix1[0, 1] = PointList[i].Y; matrix1[0, 2]
= 1;

        result = Multiplication(matrix1, matrix2);
        fP.X = (float)result[0, 0]; fP.Y = (float)result[0, 1];
        PointList[i] = fP;
    }
}
```

Данный метод записывает точку фигуры в виде матрицы из одной строки и трех столбцов, затем вызывается метод Multiplication, который умножает матрицу точки фигуры на матрицу геометрического преобразования (перемещения, поворота или масштабирования), после чего координаты точки фигуры перезаписываются и цикл повторяется.

Метод Multiplication:

```
private double[,] Multiplication(double[,] matrix1, double[,] matrix2)// умножение матриц
{
    double[,] result = new double[matrix1.GetLength(0), matrix2.GetLength(1)];
    for (int i = 0; i < matrix1.GetLength(0); i++)
    {
        for (int j = 0; j < matrix2.GetLength(1); j++)
        {
            for (int k = 0; k < matrix2.GetLength(0); k++)
            {
                result[i, j] += matrix1[i, k] * matrix2[k, j];
            }
        }
    }
    return result;
}
```

Данный метод перемножает две матрицы.

Метод Плоскопараллельное перемещение:

```
public void Move(int dx, int dy)// метод Плоскопараллельное перемещение
```

```

{
    double[,] Matrix1 = new double[1, 3];
    double[,] Matrix2 = new double[3, 3];
    Matrix2[0, 0] = Matrix2[1, 1] = Matrix2[2, 2] = 1;
    Matrix2[2, 0] = dx;
    Matrix2[2, 1] = dy;
    SameAction(Matrix1, Matrix2);
}

```

Данный метод изменяет координаты каждой точки фигуры на заданные величины.

Метод поворот:

```

public void Turn(Point P1, Point P2, Point Center)//метод поворот
{
    double alpha, al; //угол поворота
    double a, b; //a, b - радиус к первой и второй точке;
    double a2, b2, c2; //квадраты сторон
    a2 = Math.Pow(P1.X - Center.X, 2) + Math.Pow(P1.Y - Center.Y, 2);
    b2 = Math.Pow(P2.X - Center.X, 2) + Math.Pow(P2.Y - Center.Y, 2);
    c2 = Math.Pow(P2.X - P1.X, 2) + Math.Pow(P2.Y - P1.Y, 2);
    a = (Math.Sqrt(a2));
    b = (Math.Sqrt(b2));
    al = (a2 + b2 - c2) / (2 * a * b);
    if (al > 1) al = 1;
    //поиск угла поворота
    if (P1.X >= Center.X & P1.Y <= Center.Y) //1 четверть
    {
        if ((P2.X - P1.X + P2.Y - P1.Y) > 0) alpha = Math.Acos(al); //по часовой
        else alpha = -Math.Acos(al); //против часовой
    }
    else if (P1.X < Center.X & P1.Y < Center.Y) //2 четверть
    {
        if ((P2.X - P1.X) > (P2.Y - P1.Y)) alpha = Math.Acos(al); //по часовой
        else alpha = -Math.Acos(al); //против часовой
    }
    else if (P1.X <= Center.X & P1.Y >= Center.Y) //3 четверть
    {
        if ((P2.X - P1.X + P2.Y - P1.Y) < 0) alpha = Math.Acos(al); //по часовой
        else alpha = -Math.Acos(al); //против часовой
    }
    else // 4 четверть
    {
        if ((P2.X - P1.X) < (P2.Y - P1.Y)) alpha = Math.Acos(al); //по часовой
        else alpha = -Math.Acos(al); //против часовой
    }
    double[,] Matrix1 = new double[1, 3];
    double[,] Matrix2 = new double[3, 3];
    Matrix2[2, 2] = 1;
    Matrix2[0, 0] = Matrix2[1, 1] = Math.Cos(alpha);
    Matrix2[0, 1] = Math.Sin(alpha);
    Matrix2[1, 0] = -Math.Sin(alpha);
    Move(-Center.X, -Center.Y); //передвижение фигуры в координаты [0;0]
    SameAction(Matrix1, Matrix2);
    Move(Center.X, Center.Y); //передвижение фигуры обратно на то место, где она была
}

```

Данный метод осуществляет поворот фигуры на угол, вычисленный с помощью формул из тригонометрии.

Метод масштабирования по оси икс относительно центра фигуры:


```

public void Scale(Point P1, Point P2, Point Center)//метод масштабирование
{
    double[,] Matrix1 = new double[1, 3];
    double[,] Matrix2 = new double[3, 3];
    double eVector = Math.Sqrt(Math.Pow(P2.X - Center.X, 2) + Math.Pow(P2.Y -
Center.Y, 2));//длина вектора от центра фигуры к текущей позиции мышки
    double PosVector = Math.Sqrt(Math.Pow(P1.X - Center.X, 2) + Math.Pow(P1.Y -
Center.Y, 2));//длина вектора от центра фигуры к позиции, которая была до текущей позиции
мышки
    double kf = eVector / PosVector;//коэффициент масштабирования
    Matrix2[0, 0] = kf;
    Matrix2[1, 1] = 1;
    Matrix2[2, 2] = 1;
    Move(-Center.X, -Center.Y);//передвижение фигуры в координаты [0;0]
    SameAction(Matrix1, Matrix2);
    Move(Center.X, Center.Y);//передвижение фигуры обратно на то место, где она была
}

```

Данный метод осуществляет масштабирование фигуры, используя коэффициент, вычисленный с помощью длин двух векторов.

Метод отражения относительно прямой общего положения:

```

public void Mirror(PointF P, PointF F)//метод отражение
{
    float y = (P.Y * F.X - P.X * F.Y) / (F.X - P.X);//передвигаем фигуру так, чтобы
прямая проходила через начало координат
    float x = 0;
    if (y < 0)
    {
        x = (P.X * F.Y - P.Y * F.X) / (F.Y - P.Y);
        Move(-(int)x, 0);
        P.X -= x;
    }
    else
    {
        Move(0, -(int)y);
        P.Y -= y;
    }
    double rotate = -Math.Atan(P.Y / P.X);//поворачиваем фигуру так, чтобы прямая
совпала с осью икс
    double[,] Matrix1 = new double[1, 3];
    double[,] Matrix2 = new double[3, 3];
    Matrix2[2, 2] = 1;
    Matrix2[0, 0] = Matrix2[1, 1] = Math.Cos(rotate);
    Matrix2[0, 1] = Math.Sin(rotate);
    Matrix2[1, 0] = -Math.Sin(rotate);
    SameAction(Matrix1, Matrix2);
    for (int i = 0; i < PointList.Count; ++i)//отражаем фигуру по оси икс
    {
        PointF Pp = new PointF(PointList[i].X, -PointList[i].Y);
        PointList[i] = Pp;
    }
    rotate = -rotate;//поворачиваем фигуру обратно
    Matrix2[2, 2] = 1;
    Matrix2[0, 0] = Matrix2[1, 1] = Math.Cos(rotate);
    Matrix2[0, 1] = Math.Sin(rotate);
    Matrix2[1, 0] = -Math.Sin(rotate);
    SameAction(Matrix1, Matrix2);
    if (y < 0)//возвращаем фигуру обратно
    {
        Move((int)x, 0);
    }
    else

```

```
{  
    Move(0, (int)y);  
}  
}
```

Данный метод отражает фигуру следующим путем: сначала прямая движется таким образом, чтобы она проходила через начало координат, фигура движется на то же расстояние, затем прямая (и вслед за ней фигура) поворачивается на такой угол, чтобы она совпала с осью X, после чего фигура отражается относительно оси X (координата Y каждой точки меняется на противоположную), затем фигура поворачивается обратно на тот же угол и перемещается на то место, где была изначально.

6. Руководство пользователя

Программа запускается на любой версии ОС Windows (необходим установленный программный пакет Microsoft .NET Framework). Наличие среды разработки при этом не требуется - достаточно запустить исполняемый .exe файл (сразу при запуске пользователю будет предоставлена инструкция пользования программой).

Приложение после запуска выглядит следующим образом:

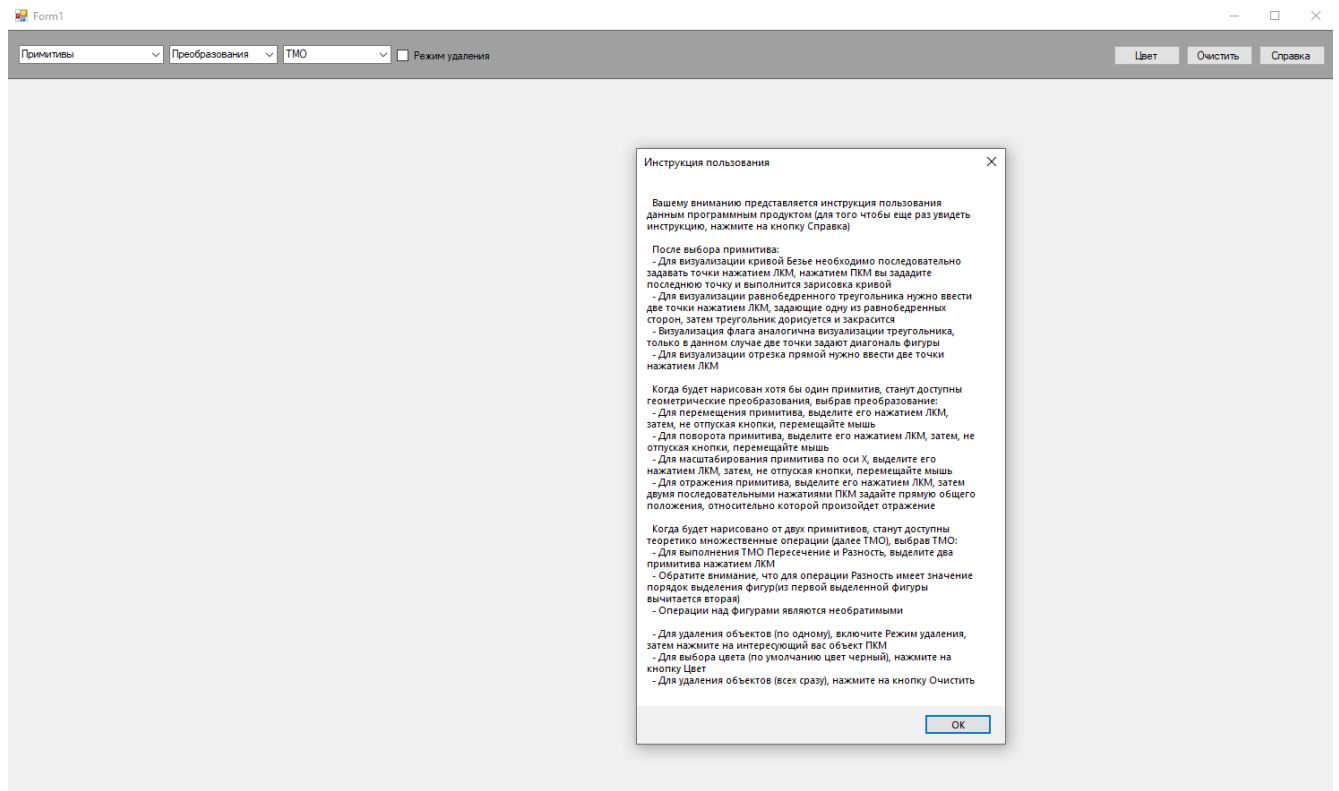


Рисунок 1 – приложение и инструкция

В верхней части графического интерфейса расположены комбо-боксы со списками примитивов, геометрических преобразований и теоретико - множественных операций. Справа от них расположен чек-бокс «Режим удаления» и три кнопки: «Цвет», для выбора цвета, «Очистить», для очистки экрана и «Справка», для того чтобы узнать как работает программа.

Закроем диалоговое окно с инструкцией и добавим несколько примитивов:

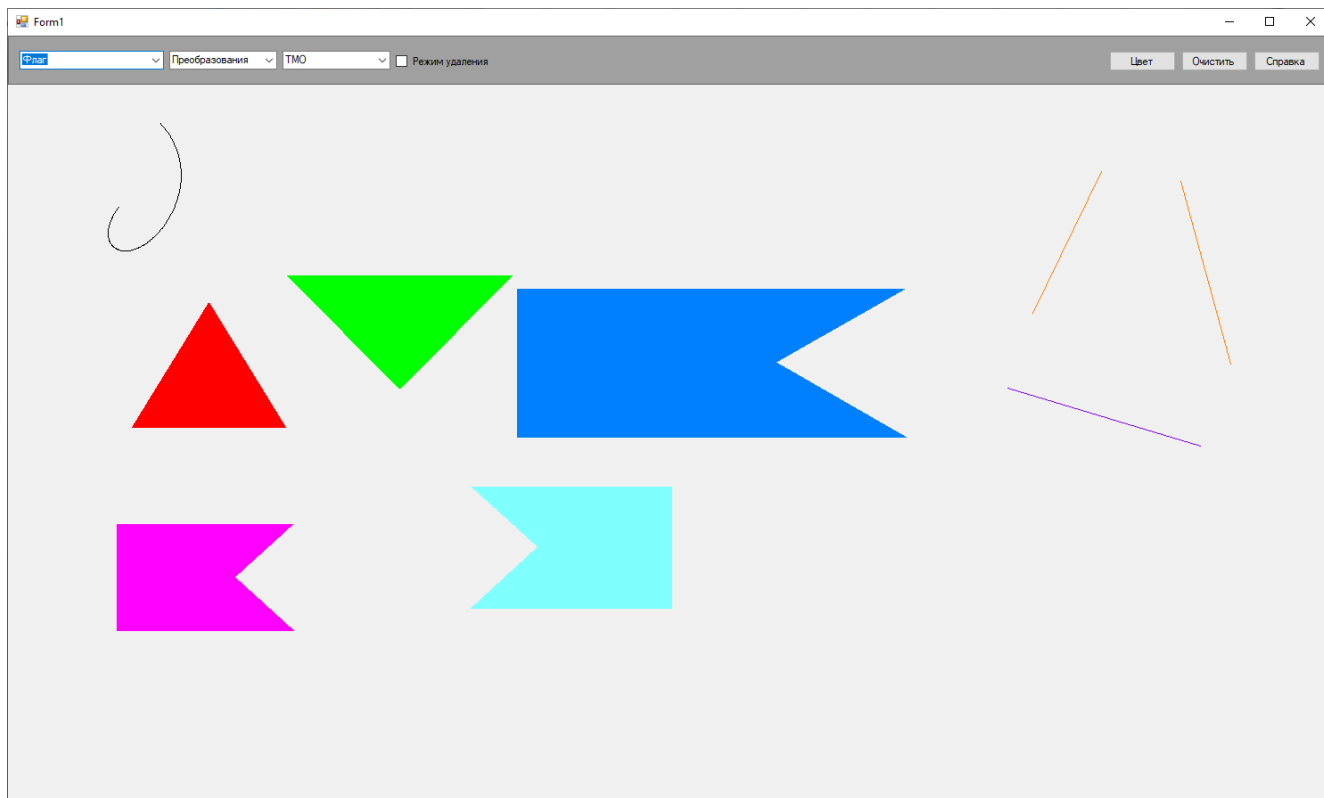


Рисунок 2 – несколько примитивов

Применим геометрические преобразования:

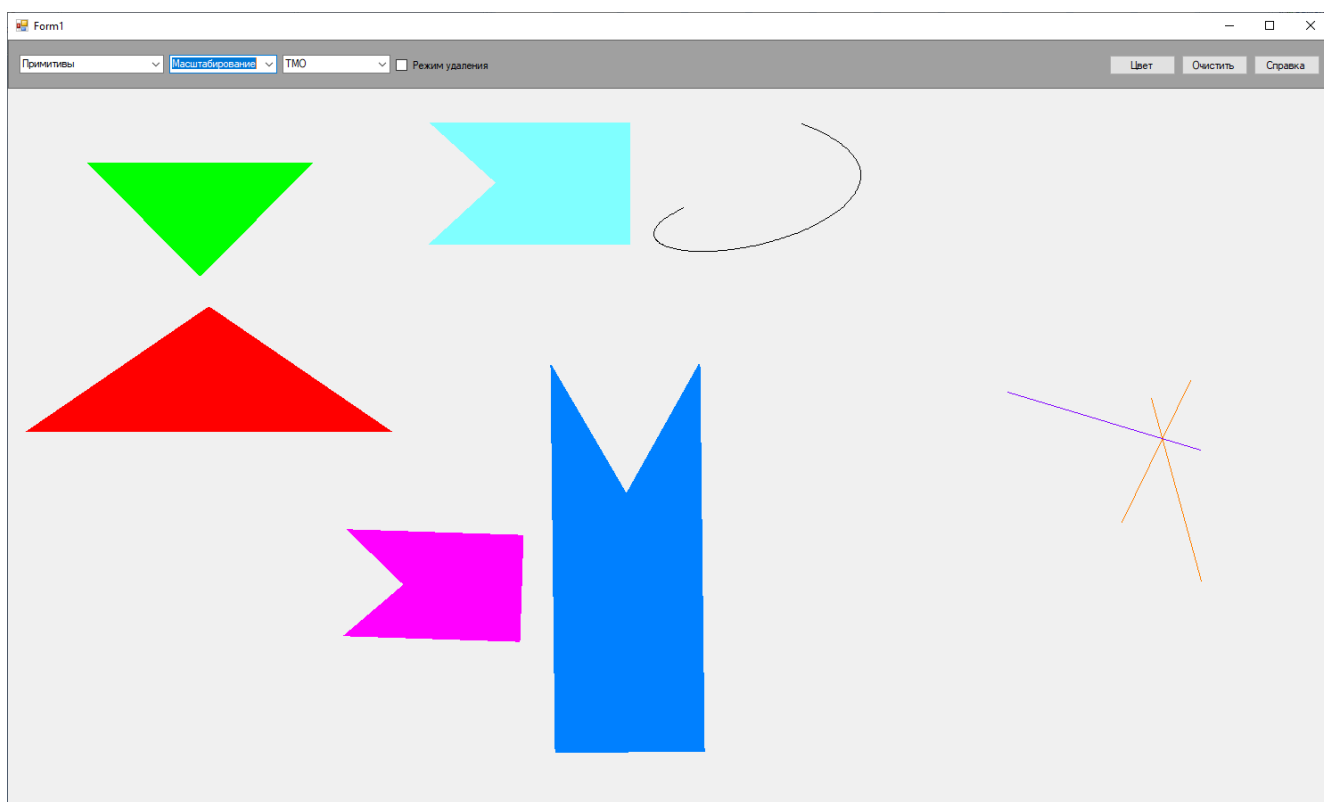


Рисунок 3 – результат геометрических преобразований.

Применим Теоретико-множественные операции:

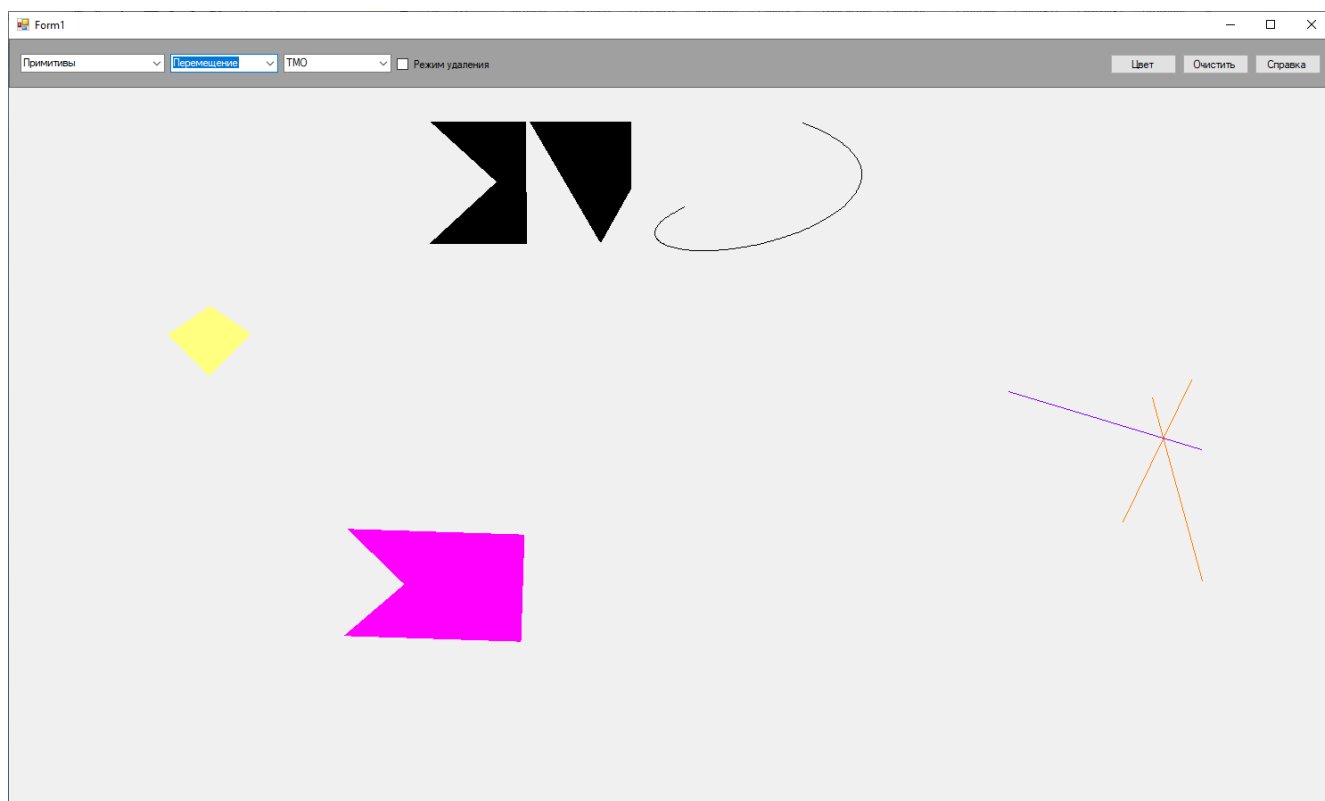


Рисунок 4 – результат теоретико-множественных операций.

Проведём геометрические преобразования с полученными фигурами:

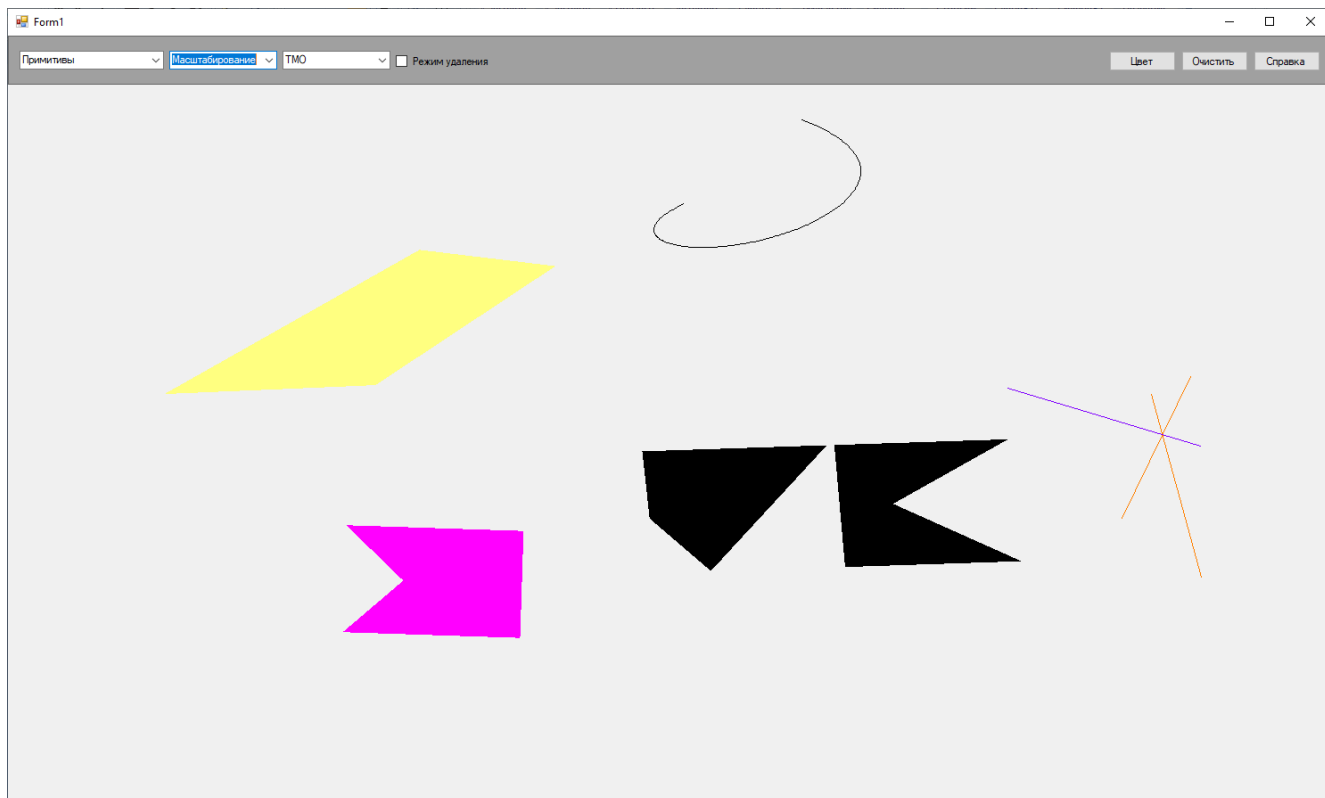


Рисунок 5 - геометрические преобразования с полученными фигурами

Библиографический список

1. Пугачев А.И. Основы компьютерной графики. Курс лекций. / Самар.гос. техн. ун-т. – Самара, 2010. – 104 с.
2. Графические системы компьютеров: методические указания к лабораторным работам и курсовому проектированию (изд. 3-е переработанное и дополненное) / А.И. Пугачев . –Самара: Самар. гос. техн. ун-т, 2021. – 110 с